

Coffee Shop Management System

Project Report & Technical Documentation

Implementation of Core Object-Oriented Programming (OOP) Principles in Java

1. Executive Summary

The Coffee Shop Management System is a Java-based application designed to simulate and automate the ordering process of a modern coffee shop. The application allows users to view an interactive menu containing hot drinks, cold drinks, and special items, customize orders, and generate a final itemized receipt.

The primary objective of this project is to demonstrate a clean, robust architecture adhering strictly to the four fundamental pillars of Object-Oriented Programming (OOP) in Java: Abstraction, Encapsulation, Inheritance, and Polymorphism.

2. Core OOP Principles Implementation & Code Breakdown

A. Abstraction: Abstract classes define abstract concepts that cannot be instantiated directly. In this project, 'Drink' represents an abstract concept since customers order specific beverage types (Hot, Cold, Special), not a generic 'Drink'.

Class 'Drink' is declared as 'abstract', preventing direct instantiation ('new Drink()'). It defines the abstract method 'public abstract String getInfo()', enforcing every concrete subclass to provide its specific implementation details.

Code Snippet: Drink.java (Abstraction)

```
public abstract class Drink {
    private String name;
    private double price;

    public Drink(String name, double price) {
        this.name = name;
        this.price = price;
    }

    public String getName() { return name; }
    public double getPrice() { return price; }

    // ABSTRACT METHOD: Enforces polymorphism and subclass formatting
    public abstract String getInfo();
}
```

```
}
```

B. Encapsulation: Encapsulation restricts direct access to an object's internal components, preventing unauthorized modifications and safeguarding data integrity.

All instance variables across all classes (e.g., 'name', 'price', 'drinks') are marked 'private'. Access is granted strictly via getter and setter methods. Furthermore, in 'Menu.java', 'getDrinks()' returns an unmodifiable list ('Collections.unmodifiableList') to prevent external code from tampering with the menu list directly.

Code Snippet: Menu.java (Encapsulation)

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public class Menu {
    private final ArrayList<Drink> drinks = new ArrayList<>();

    public void addDrink(Drink drink) {
        drinks.add(drink);
    }

    // ENCAPSULATION: Returns an unmodifiable view to protect the list
    public List<Drink> getDrinks() {
        return Collections.unmodifiableList(drinks);
    }

    public void showMenu() {
        System.out.println("\n---- Coffee Shop Menu ----");
        for (int i = 0; i < drinks.size(); i++) {
            System.out.println((i + 1) + ". " + drinks.get(i).getInfo());
        }
    }
}
```

C. Inheritance: Inheritance enables new classes to derive properties and behaviors from an existing class, facilitating code reusability and hierarchical organization.

The subclasses 'HotDrink', 'ColdDrink', and 'SpecialDrink' extend the abstract parent class 'Drink' using the 'extends' keyword. Each subclass inherits the base attributes ('name', 'price') and adds specialized attributes (e.g., 'size' for hot drinks, 'hasIce' for cold drinks).

Code Snippet: HotDrink.java (Inheritance)

```

public class HotDrink extends Drink {
    private String size; // Small, Medium, Large

    public HotDrink(String name, double price, String size) {
        super(name, price); // INHERITANCE: Calls parent constructor
        this.size = size;
    }

    public String getSize() { return size; }

    @Override
    public String getInfo() {
        return "[Hot] " + getName() + " (" + size + ") - " + getPrice() + " SAR";
    }
}

```

Code Snippet: ColdDrink.java (Subclassing)

```

public class ColdDrink extends Drink {
    private boolean hasIce;

    public ColdDrink(String name, double price, boolean hasIce) {
        super(name, price);
        this.hasIce = hasIce;
    }

    @Override
    public String getInfo() {
        String iceText = hasIce ? "With Ice" : "No Ice";
        return "[Cold] " + getName() + " (" + iceText + ") - " + getPrice() + " SAR";
    }
}

```

D. Polymorphism: Polymorphism allows objects of different classes to be treated as objects of a common superclass, dynamically executing overridden behaviors at runtime.

The system stores heterogeneous objects ('HotDrink', 'ColdDrink', 'SpecialDrink') inside a unified collection 'ArrayList<Drink>'. When 'getInfo()' or 'getPrice()' is invoked in 'Order.java' or 'Menu.java', Java dynamically executes the specific implementation for that exact object type at runtime (Dynamic Binding).

Code Snippet: Order.java & Main Application (Polymorphism)

```

import java.util.ArrayList;

public class Order {

```

```

private Customer customer;
private ArrayList<Drink> orderedDrinks;

public Order(Customer customer) {
    this.customer = customer;
    this.orderedDrinks = new ArrayList<>();
}

public void addDrink(Drink drink) {
    orderedDrinks.add(drink);
}

public double calculateBill() {
    double total = 0;
    for (Drink d : orderedDrinks) {
        total += d.getPrice(); // POLYMORPHISM
    }
    return total;
}

public void printOrder() {
    System.out.println("\n---- Order Summary ----");
    System.out.println("Customer: " + customer.getName());
    for (Drink d : orderedDrinks) {
        System.out.println("- " + d.getInfo()); // POLYMORPHISM: Dynamic Method
Dispatch
    }
    System.out.println("Total: " + calculateBill() + " SAR");
}
}

```

3. Project Architecture & Class Descriptions

Class Name	Class Type	Role & Responsibility in Project
Drink	Abstract Class	Parent class defining common beverage properties (name, price) and abstract method getInfo().
HotDrink	Concrete Subclass	Inherits from Drink; represents hot beverages and manages cup size attributes.
ColdDrink	Concrete Subclass	Inherits from Drink; represents cold beverages and tracks ice preference.

SpecialDrink	Concrete Subclass	Inherits from Drink; represents signature beverages with custom formatting.
Customer	Class	Encapsulates customer details (e.g., customer name).
Menu	Class	Manages the beverage menu collection with encapsulated read-only list access.
Order	Class	Handles customer order items, total bill calculation, and receipt printing.
CoffeeSystem	Main Class	Main entry point containing UI execution loop and user interaction logic.

4. Conclusion

The Coffee Shop Management System successfully demonstrates how object-oriented design enhances software modularity, security, and extensibility. By leveraging Abstraction, Encapsulation, Inheritance, and Polymorphism, the application achieves a clean architecture where new beverage types or system features can be added seamlessly without altering existing core logic.